PyTorch 编译流程及优化整理

PyTorch → TorchDynamo → AOT Autograd → Inductor → Triton Kernel

一、TorchDynamo

对网络进行图捕获：将要执行的 Python 函数的字节码，在翻译字节码的过程中构建 FX Graph

二、AOT Autograd 前

在 AOT Autograd 前运行 pre_grad_passes 对捕获到的图进行优化：

- sink_cat_after_pointwise：将 cat 算子移植一元 pointwise 算子 relu、tanh 之后
- permute_fusion：对 linear/matmul/bmm+ permute/transpose 的算子组合进行垂直融合
- batch_fusion：对符合条件的 torch.nn.functional.linear、layer_norm、tanh 算子融合为 torch.baddmm 等进行批处理算子
- pattern_matcher：normalization_pass、merge_splits_pass、split_cat_pass、unbind_stack_pass

三、AOT Autograd

AOT Autograd 在执行正向传播的过程中动态构建反向传播计算图，得到联合计算图(joint forward and backward graph)，会进行 joint_graph_passes 优化：

- constant_fold_uniform_value
- pattern_matcher
- replace_random_passes

AOT Autograd 会将复杂的算子(特别是自定义的或高阶的算子)分解成一系列简单的、更基础的算子

AOTAutograd 会将 joint graph 划分为 foward graph 和 backward graph，可以通过求解最大流/最小割问题最小化正向传播保留给反向传播的张量

四、AOT Autograd 后

在 AOT Autograd 后运行 post_grad_passes 优化：

- eliminate_dead_code：死代码消除
- reorder_for_locality：通过将生产者节点移至消费者节点附近来提升内存局部性
- group_batch_fusion_post_grad_passes：将多个 mm 和 addmm 操作通过 torch.ops.fbgemm.gmm 实现融合
- remove_extra_clones：移除不必要的 clone 算子

- pattern_matcher

TorchInductor Example

Input Code	ATen FX Graph	Define-by-run IR	Scheduling/Fusion	Output Triton	Output Wrapper

```python
import torch

@torch.compile(dynamic=True)
def toy_example(x):
    y = x.sin()
    z = y.cos()
    return y, z


toy_example(torch.randn([8192, 1024], device="cuda"))
```

Run with:
TORCH_COMPILE_DEBUG=1 python inductor_demo.py

TorchInductor Example

Input Code	ATen FX Graph	Define-by-run IR	Scheduling/Fusion	Output Triton	Output Wrapper

```python
def forward(self, arg0_1: f32[s0, s1]):
    # File: inductor_demo.py:6, code: y = x.sin()
    sin: f32[s0, s1] = torch.ops.aten.sin.default(arg0_1)

    # File: inductor_demo.py:7, code: z = y.cos()
    cos: f32[s0, s1] = torch.ops.aten.cos.default(sin)
    return (sin, cos)
```

五、GraphLowering

把计算图转为 TorchInductor IR 生成 buf 阶段也会对一些算子融合到一个 buf 中

TorchInductor Example

Input Code	ATen FX Graph	**Define-by-run IR**	Scheduling/Fusion	Output Triton	Output Wrapper

```python
def inner_fn_buf0(index):
  i0, i1 = index
  tmp0 = ops.load(arg0_1, i1 + i0 * s1)
  tmp1 = ops.sin(tmp0)
  return tmp1

def inner_fn_buf1(index):
  i0, i1 = index
  tmp0 = ops.load(buf0, i1 + i0 * s1)
  tmp1 = ops.cos(tmp0)
  return tmp1
```

```python
buf0_ir = TensorBox(StorageBox(ComputedBuffer(
  name='buf0',
  layout=FixedLayout('cuda', torch.float32,
                     size=[s0, s1], stride=[s1, 1]),
  data=Pointwise(inner_fn=inner_fn_buf0,
                 ranges=[s0, s1], ...))))

buf1_ir = TensorBox(StorageBox(ComputedBuffer(
  name='buf1',
  layout=FixedLayout('cuda', torch.float32,
                     size=[s0, s1], stride=[s1, 1]),
  data=Pointwise(inner_fn=inner_fn_buf1,
                 ranges=[s0, s1], ...))))
```

六、Scheduler 阶段

进行 buf 间的调度融合：

- 当前 Inductor 融合判别规则中严格要求两 buf 的读写存在交集，对于其中 buf 名称相同但是偏移索引不同的情况也视为不存在交集。
- aggressive_fusion 选项控制（默认关闭）：将水平融合的判断提前到有无共享数据判断之前，实现 Kernel 间的水平融合（将 Kernel 的多次调用转换为一次调用）
- epilogue_fusion 选项控制（默认开启）：融合 pointwise 至模版算子

TorchInductor Example

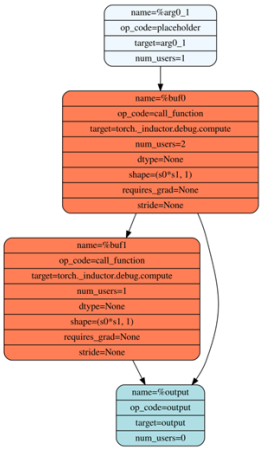
Input Code	ATen FX Graph	Define-by-run IR	**Scheduling/Fusion**	Output Triton	Output Wrapper

torch/_inductor/scheduler.py

Scheduler.can_fuse(buf0, buf1)
True

Scheduler.score_fusion(buf0, buf1)
(True, True, 33554432, -1)

- True/True is category of fusion (pointwise+pointwise)
- 33554432 is estimated memory bandwidth saved by fusion: 8192*1024*4
- -1 is distance in input graph



七、Triton Kernel 生成

对 ScheduleNode 进行 Triton Kernel 生成。

TorchInductor Example

Input Code	ATen FX Graph	Define-by-run IR	Scheduling/Fusion	**Output Triton**	Output Wrapper

```
@triton.jit
def triton__0(in_ptr0, out_ptr0, out_ptr1, xnumel, XBLOCK : tl.constexpr):
    xoffset = tl.program_id(0) * XBLOCK
    xindex = xoffset + tl.arange(0, XBLOCK)[:]
    xmask = xindex < xnumel
    x0 = xindex
    tmp0 = tl.load(in_ptr0 + (x0), None)
    tmp1 = tl.sin(tmp0)
    tmp2 = tl.cos(tmp1)
    tl.store(out_ptr0 + (x0 + tl.zeros([XBLOCK], tl.int32)), tmp1, None)
    tl.store(out_ptr1 + (x0 + tl.zeros([XBLOCK], tl.int32)), tmp2, None)
```

TorchInductor Example

Input Code	ATen FX Graph	Define-by-run IR	Scheduling/Fusion	Output Triton	**Output Wrapper**

```
def call(args):
    arg0_1, = args
    args.clear()
    arg0_1_size = arg0_1.size()
    s0 = arg0_1_size[0]
    s1 = arg0_1_size[1]
    buf0 = empty_strided((s0, s1), (s1, 1), device='cuda', dtype=torch.float32)
    buf1 = empty_strided((s0, s1), (s1, 1), device='cuda', dtype=torch.float32)
    triton__0_xnumel = s0*s1
    triton__0.run(arg0_1, buf0, buf1, triton__0_xnumel, grid=grid(triton__0_xnumel))
    return (buf0, buf1, )
```

综合上述流程中的图融合优化，其中 Inductor 编译器所做的图融合可以概括为以下几个类别：

Pointwise 融合：如果多个 Pointwise 算子的 shape 相同，会在 Graph Lowering 阶段被融合在一起，表现为所有的 Pointwise 算子都出现在同一个 Pointwise IR buf 节点中。

view 移除：view 算子通常不需要代码生成，可以对其进行移除后，只需修改后续访问张量的索引表达式，就可以直接索引到原始输出。

模式匹配：针对 FX graph 的优化 pass，会寻找匹配相应的 FX 节点的模式，满足条件后用单个算子进行替换。

尾操作融合：针对矩阵乘、卷积等使用 Triton 模板生成的 Kernel，可以将额外的 Point wise 算子融合到该 Kernel 的尾处理操作中。但尾操作融合并不总是有益的，因为像矩阵乘法这样的复杂 Kernel 通常需要进行微调，添加额外的计算可能会使它们的性能相较未融合前更差。

垂直融合：如果消费者中的所有读取都匹配生产者中的相应写入，或者是可以由两个节点融合之前融合调度的节点写入，则可以将消费者融合到生产者中。

水平融合：两个不相互依赖但共享读取的节点也可以融合。

PyTorch 编译流程代码解析

深度学习执行模式包括 Eager（define by run）和 Graph（define and run），PyTorch2.0 将两种模式结合，既保留了灵活性，又能够通过优化计算图来提升性能。

早期图捕获技术包括torch.jit.trace、torch.jit.script、Lazy Tensors、torch.fx.symbolic_trace、torch.onnx.export、JAX 等，这些技术针对不同的场景提供了不同的解决方案。PyTorch2.0 中的 TorchDynamo 和 TorchInductor 综合了这些技术的优点，通过动态字节码转换和图编译，提供了更为灵活和高效的图捕获和执行方式。

TorchDynamo 通过 torch.compile 接口接收一个 torch.fx.Graph 模型和示例输入后，会将其编译为高效执行的版本，其核心在于通过动态编译、符号化分析和控制流特化，确保对复杂模型的高效执行，同时保持了 PyTorch 的灵活性和易用性。通过 TorchDynamo，PyTorch2.0 实现了对动态 Python 字节码的高效捕获和转换，极大提升了模型的训练和推理性能。

TorchInductor 是 PyTorch 的编译器后端，与 TorchDynamo 配合使用，可以将捕获的 FX 图转换为高效的代码。TorchInductor 的设计高度兼容 PyTorch，采用了与之类似的抽象，允许支持所有 PyTorch 功能，且使用 Python 实现，易于修改和理解；TorchInductor 并未专注优化一小部分已经广泛研究的经典模型，而是早期就着重支持多种算子、硬件和优化技术，使其能够更通用，能扩展到多种场景。在输出语言方面，TorchInductor 借鉴了 PyTorch 用户编写高性能内核的方式，重用先进的编程语言，高性能的 CPU 内核通常用 C++/OpenMP 编写，GPU 内核生成 Triton 和 C++代码。其中 Triton 对 cuda 体系进行了抽象，Tensor 的加载被转换成向量化处理，kernel 一次处理 XBLOCK 个元素，如果元素数量不是 XBLOCK 的倍数，某些元素可能会在末尾被 mask 掉；对于 CPU 后端，TorchInductor 生成带有 OpenMP 的 C++代码。C++后端有 vectorize 版本和 non-vectorize 版本两个变体。

TorchInductor 通过 decompositions 技术（@register_decomposition），没有为所有 PyTorch Op 实现向其 IR 的 Lowering，而是使用 AOTAutograd 将许多 Op 分解为一组更简单的 Op，分解的过程可以递归调用，在此过程中可能会触发其他算子的拆分函数，直到达到终态。因此可以推导出来，但分解的过程不能包含循环。

TorchInductor 的 scheduling 包括算子融合、Kernel 顺序调度和内存规划，通过 SchedulerNode 表示要进行代码生成的 kernel。算子融合的关键在于确保内存读写顺序的正确性，并通过打分函数评估融合的有效性。在算子融合过程中，有两个关键的函数：

can_fuse：考虑内存读写顺序、依赖关系等融合合法性来判断两算子是否能够融合，且融合是硬件相关的，如 Triton GPU 支持 reduce-broadcast-reduce；

score_function：考虑内存节省、计算复杂度和硬件支持等因素来评估融合的收益和成本，具体地，该函数会评估 fusion 类型（pointwise、reduction、template 等）、可节省的内存大小、节点间的最短距离等。

此外，PyTorch 2.0 的 TorchInductor 提供了强大的功能来处理动态形状，使得模型能够在不同输入数据上高效运行。

使用 PyTorch 的 Python 代码在 torch.compile 后，Torch Dynamo 会解释运行 Python Bytecode，捕捉跟踪到对应的 FX Graph，AOTAutograd 也会根据该前向 FX Graph 生成反向 FX Graph，进而交给 Inductor。

Python 代码

```
def forward(self, arg0_1: "f32[32, 512, 1024]"):
    floor : "f32[32, 512, 1024]" = torch.ops.aten.floor.default(arg0_1)
    ceil  : "f32[32, 512, 1024]" = torch.ops.aten.ceil.default(arg0_1);     arg0_1 = None
    add   : "f32[32, 512, 1024]" = torch.ops.aten.add.Tensor(floor, ceil);  floor = ceil = None
    sum_1 : "f32[32, 512]"       = torch.ops.aten.sum.dim_IntList(add, [-1]);add = None
    add_1 : "f32[32, 512]"       = torch.ops.aten.add.Tensor(sum_1, 1);       sum_1 = None
    return (add_1,)
```

Inductor 不强行追求完备性以覆盖所有 Op，而是通过 fallback 一些算子，优化一些算子，将 aten op 进行拆分后降级为 IR。

Op 降级至 Inductor IR 的主要逻辑由 make_pointwise 和 make_reduction 完成，例如下图 ceil 和 sum 的调用示例，ceil 中调用了 make_pointwise(fn)(x)，sum 中也调用了 make_reduction。

```
@register_lowering(aten.ceil)
def ceil(x):
    if is_integer_type(x):
        return clone(x)
    fn = ops_wrapper("ceil")
    return make_pointwise(fn)(x)
```

```
@register_lowering([aten.sum, prims.sum])
def sum_(x, axis=None, keepdims=False, *, dtype=None):
    if (
        is_integer_dtype(x.get_dtype()) or is_boolean_dtype(x.get_dtype())
    ) and dtype is None:
        dtype = torch.int64

    fn = make_reduction("sum", override_return_dtype=dtype)
    return fn(x, axis, keepdims, dtype=dtype)
```

```
def make_pointwise(
    fn,
    override_return_dtype=None,
    override_device=None,
    override_fn_when_input_bool=None,
    override_fn_when_cuda_float64=None,
    allow_alpha=False,
    triton_fallback=None,
):
    def inner(*inputs: List[TensorBox], alpha=None):
        ... #处理triton_fallback和allow_alpha
        loaders = [x.make_loader() for x in inputs]
        ranges = inputs[0].get_size()
        dtype = override_return_dtype or inputs[0].get_dtype()
        is_cuda = decode_device(inputs[0].get_device()).type == "cuda"
        ... #检查inputs的size
        def inner_fn(index):
            assert len(index) == len(ranges), f"wrong ndim {index} {ranges}"
            if dtype == torch.bool and override_fn_when_input_bool is not None:
                return override_fn_when_input_bool(*[load(index) for load in loaders])
            elif override_fn_when_cuda_float64 and is_cuda and dtype == torch.float64:
                return override_fn_when_cuda_float64(*[load(index) for load in loaders
            else:
                return fn(*[load(index) for load in loaders])
        ... #处理override_device
        return Pointwise.create(
            device=device,
            dtype=dtype,
            inner_fn=inner_fn,
            ranges=ranges,
        )

    return inner
```

Inductor IR

FX Graph Node 与 IR 中的 TensorBox 相对应，inner_fn 可以算 Node 的计算结果，即 out[i0,i1,i2]=inner_fn([i0,i1,i2])，特别地，由于 Node 无法存储，add Node 的 inner_fn 将计算进行了 fuse，要存储的计算结果在 IR 中是 ComputedBuffer，示例中存储到了 buf0 中，进而在后续 Node 中不需要在 inner_fn 中计算，直接使用 buf0 即可，此时计算类型由 Pointwise 变为 Reduction，即 out[i0,i1]=sum(inner_fn([i0,i1],[r0]) for r0 in range(1024))，张量 Shape 由 32*512*1024 归约为 32*512，最后的输出 Node，结果需要放入内存，存到 buf1 中。

```
Node(
    name="arg0_1",
    op="placeholder",
    target="arg0_1",
    args=(),
    users={floor: None, ceil: None},
    meta={tensor_meta: TensorMetadata(
        shape=torch.Size([32, 512, 1024]),
        dtype=torch.float32,
        requires_grad=False,
        stride=(524288, 1024, 1),
        memory_format=torch.contiguous_format,
    )}
)

TensorBox(StorageBox(
    InputBuffer(
        name='arg0_1',
        layout=FixedLayout(
            'cuda:0',
            torch.float32,
            size=[32, 512, 1024],
            stride=[524288, 1024, 1]
        )
    )
))
```

```
Node(
    name="floor",
    op="call_function",
    target=OpOverload(op='aten.floor', overload='default'),
    args=(arg0_1,),
    users={add: None},
    meta={...}
)

TensorBox(StorageBox(
  Pointwise(
    'cuda:0',
    torch.float32,
    def inner_fn(index):
        i0, i1, i2 = index
        tmp0 = ops.load(arg0_1, i2 + 1024 * i1 + 524288 * i0)
        tmp1 = ops.floor(tmp0)
        return tmp1
    ,
    ranges=[32, 512, 1024],
    origin_node=None,
    origins={floor}
  )
))
```

```
Node(
    name="ceil",
    op="call_function",
    target=OpOverload(op='aten.ceil', overload='default'),
    args=(arg0_1,),
    users={add: None},
    meta={...}
)

TensorBox(StorageBox(
  Pointwise(
    'cuda:0',
    torch.float32,
    def inner_fn(index):
        i0, i1, i2 = index
        tmp0 = ops.load(arg0_1, i2 + 1024 * i1 + 524288 * i0)
        tmp1 = ops.ceil(tmp0)
        return tmp1
    ,
    ranges=[32, 512, 1024],
    origin_node=None,
    origins={ceil}
  )
))
```

```
Node(
    name="add",
    op="call_function",
    target=OpOverload(op='aten.add', overload='Tensor'),
    args=(floor,ceil),
    users={sum_1: None},
    meta={...}
)

TensorBox(StorageBox(
  Pointwise(
    'cuda:0',
    torch.float32,
    def inner_fn(index):
        i0, i1, i2 = index
        tmp0 = ops.load(arg0_1, i2 + 1024 * i1 + 524288 * i0)
        tmp1 = ops.floor(tmp0)
        tmp2 = ops.load(arg0_1, i2 + 1024 * i1 + 524288 * i0)
        tmp3 = ops.ceil(tmp2)
        tmp4 = tmp1 + tmp3
        return tmp4
    ,
    ranges=[32, 512, 1024],
    origin_node=None,
    origins={floor, ceil, add}
  )
))
```

```
Node(
    name="sum_1",
    op="call_function",
    target=OpOverload(op='aten.sum', overload='dim_IntList'),
    args=(add,[-1]),
    users={add_1: None},
    meta={...}
)

TensorBox(StorageBox(
  ComputedBuffer(
    name='buf0',
    layout=FlexibleLayout('cuda:0', torch.float32, size=[32, 512], stride=[512, 1]),
    data=Reduction(
      'cuda:0',
      torch.float32,
      def inner_fn(index, rindex):
          i0, i1 = index
          r0 = rindex
          tmp0 = ops.load(arg0_1, r0 + 1024 * i1 + 524288 * i0)
          tmp1 = ops.floor(tmp0)
          tmp2 = ops.load(arg0_1, r0 + 1024 * i1 + 524288 * i0)
          tmp3 = ops.ceil(tmp2)
          tmp4 = tmp1 + tmp3
          return tmp4
      ,
      ranges=[32, 512],
      reduction_ranges=[1024],
      reduction_type=sum,
      origin_node=None,
      origins={floor, ceil, sum_1, add}
    ))
))
```

```
Node(
    name="add_1",
    op="call_function",
    target=OpOverload(op='aten.add', overload='Tensor'),
    args=(sum_1, 1),
    users={output: None},
    meta={...}
)

TensorBox(StorageBox(
  Pointwise(
    'cuda:0',
    torch.float32,
    def inner_fn(index):
        i0, i1 = index
        tmp0 = ops.load(buf0, i1 + 512 * i0)
        tmp1 = ops.constant(1, torch.float32)
        tmp2 = tmp0 + tmp1
        return tmp2
    ,
    ranges=[32, 512],
    origin_node=None,
    origins={add_1}
  )
))
```

```
Node(
    name="output",
    op="output",
    target="output",
    args=((add_1,),),
    users={},
    meta={...}
)

TensorBox(StorageBox(
  ComputedBuffer(
    name='buf1',
    layout=FixedLayout('cuda:0', torch.float32, size=[32, 512], stride=[512, 1]),
    data=Pointwise(
      'cuda:0',
      torch.float32,
      def inner_fn(index):
        i0, i1 = index
        tmp0 = ops.load(buf0, i1 + 512 * i0)
        tmp1 = ops.constant(1, torch.float32)
        tmp2 = tmp0 + tmp1
        return tmp2
      ,
    ranges=[32, 512],
    origin_node=add_1,
    origins={add_1}
  ))
))
```

除 Pointwise 和 Reduction 外还有其他 IR 定义，包括 Scan、Scatter、View、ExternKernel、TritonTemplateKernel，其中 ExternKernel、TritonTemplateKernel 可用于表达矩阵乘法。

ExternKernel 是直接调用 BLAS 库中优化好的 Kernel，TensorBox 中没有计算逻辑，仅记录了 Python Wrapper 和 Cpp Wrapper 中要跑的函数，但这种方法的问题是使得矩阵乘法 Kernel 无法与后续的操作进行融合。

```
TensorBox(StorageBox(
  ExternKernelOut(
    python_kernel_name='extern_kernels.mm',
    name=buf0,
    layout=FlexibleLayout('cuda:0', torch.float32, size=[32, 1024], stride=[1024, 1]),
    inputs=[InputBuffer(name='arg0_1',
                        layout=FixedLayout('cuda:0', torch.float32, size=[32, 1024], s
            InputBuffer(name='arg1_1',
                        layout=FixedLayout('cuda:0', torch.float32, size=[1024, 1024],
    constant_args=(),
    kwargs={},
    output_view=None,
    python_kernel_name=extern_kernels.mm,
    cpp_kernel_name=at::mm_out,
    ordered_kwargs_for_cpp_kernel=(),
    origin_node=None,
    origins={mm}
  )
))
```

TritonTemplateKernel 利用 OpenAI Triton 可以解决上述问题，但在 TensorBox 中没有 Triton 代码，提供 make_kernel_render 函数在后续导出 Triton 代码。

```
TensorBox(StorageBox(
  TritonTemplateBuffer(
    name='buf0',
    layout=FixedLayout('cuda:0', torch.float32, size=[32, 1024], stride=[1024, 1])),
    make_kernel_render=...
))
```

从 Inductor IR 生成 Triton Kernel

Inductor IR 里的 def inner_fn，实际上是由一堆 make_xxx 里的 inner_fn，和另一堆 IRNode subclass 里的 loader 拼出来的。Inductor 在生成 Triton Kernel 时先是在 LoopBodyBlock 里，把 inner_fn 转换成一张 FX Graph，并且在图的最后补上了 inner_fn 一直没有的 ops.store。

torch.compile 的产物在/tmp/torchinductor_username/*里，其中一个 py 文件里只有一个 triton kernel，另一个 py 文件负责编译和运行 Triton Kernel，Inductor Op 到 Triton Op 之间的转换都是在 triton.py 里定义的。

Inductor 生成的@pointwise 就起到了@triton.autotune 的效果，生成了一系列 block size，然后选择跑得最快的一个 block size。Inductor 提供 block size 的方法相当简单粗暴，对于不同类型的计算有一套固定的 block size。如果 numel 足够大，一维 Pointwise 的 XBLOCK 基本就是 1024 或者 512。在 triton_config_with_settings，Inductor 还会考虑 grid 能不能覆盖整个 tensor。不过考虑到 N 卡上 tl.program_id(0)最大到 2147483647，这些调整基本都不会发生。

reduction 类型要在最后一个 dimension 上求和，那自然不能像 pointwise 的 Triton Kernel 一样全部拍平，生成的 Triton Kernel 先是 load 了一整行（[XBLOCK=1，RBLOCK=1024]），经过同样的计算后（tmp123），多进行了一次 tl.sum（tmp7）计算一行的 sum。

如果最后一维再小一些，输入就非常小（[XBLOCK，RBLOCK=16]），有的 CUDA Core 就只能摸鱼了。这个时候 XBLOCK 就不是固定的 1，而是由 persistent_reduction 在[1,8,32,128]里挑一个最快的当 XBLOCK。

如果最后一维再大一些，直接 load 一整行就不可行了。最坏情况下，load 的结果就可

能用完 Shared Memory。所以这种情况下 Inductor 生成的 Triton Kernel 里会有一个 for loop。循环里每次 load 并且 accumulate[XBLOCK, RBLOCK]大小的块，最后输出前再做最后的 sum（[XBLOCK, RBLOCK] -> [XBLOCK]）。

persistent_reduction 没有 for，后面一个 reduction 有 for，是在 codegen_body（/torch/_inductor/codegen/triton.py:L2446）里做了个特判

```
@torch.compile
def fb(x: torch.Tensor):
    a = torch.floor(x) + torch.ceil(x)
    b = a.sum(dim=-1)
    return b
fb(torch.empty([32,512,1024], device="cuda"))
```

```
ComputedBuffer(
    name='buf0',
    layout=FixedLayout('cuda', torch.float32, size=[32, 512], stride=[512, 1]),
    data=Reduction(
        'cuda',
        torch.float32,
        def inner_fn(index, rindex):
            i0, i1 = index
            r0 = rindex
            tmp0 = ops.load(arg0_1, r0 + 1024 * i1 + 524288 * i0)
            tmp1 = ops.floor(tmp0)
            tmp2 = ops.load(arg0_1, r0 + 1024 * i1 + 524288 * i0)
            tmp3 = ops.ceil(tmp2)
            tmp4 = tmp1 + tmp3
            return tmp4
        ,
        ranges=[32, 512],
        reduction_ranges=[1024],
        reduction_type=sum,
        origin_node=sum_1,
        origins={sum_1, floor, ceil, add}
))
```

```
#fb(torch.empty([32,512,32768], device="cuda"))
@reduction(
    size_hints=[16384, 32768],
    reduction_hint=ReductionHint.INNER,
    filename=__file__,
    meta={'signature': {0: '*fp32', 1: '*fp32', 2: 'i32', 3: 'i32'}, 'device': 0, 'dev
)
@triton.jit
def triton_(in_ptr0, out_ptr0, xnumel, rnumel, XBLOCK : tl.constexpr, RBLOCK : tl.cons
    xnumel = 16384
    rnumel = 32768
    xoffset = tl.program_id(0) * XBLOCK
    xindex = xoffset + tl.arange(0, XBLOCK)[:, None]
    xmask = xindex < xnumel
    rbase = tl.arange(0, RBLOCK)[None, :]
    x0 = xindex
    _tmp5 = tl.full([XBLOCK, RBLOCK], 0, tl.float32)
    for roffset in range(0, rnumel, RBLOCK):
        rindex = roffset + rbase
        rmask = rindex < rnumel
        r1 = rindex
        tmp0 = tl.load(in_ptr0 + (r1 + (32768*x0)), rmask, other=0)
        tmp1 = tl.math.floor(tmp0)
        tmp2 = tl.math.ceil(tmp0)
        tmp3 = tmp1 + tmp2
        tmp4 = tl.broadcast_to(tmp3, [XBLOCK, RBLOCK])
        tmp6 = _tmp5 + tmp4
        _tmp5 = tl.where(rmask, tmp6, _tmp5)
    tmp5 = tl.sum(_tmp5, 1)[:, None]
    tl.store(out_ptr0 + (x0), tmp5, None)
```

Schedule：

GraphLowering 的过程里，一个函数经常会被拆成好几个 IR，包括 ComputedBuffer、TemplateBuffer、ExternKernelOut，这时候就需要 Scheduler 把 IR 生成的 Kernel 串起来或者拼起来。

Scheduler 的逻辑基本都在自己的__init__里。拿到 GraphLowering 给的 IR 后，Scheduler 先会把 IR 套到 SchedulerNode 里，然后 Scheduler 会根据 IR 里的信息，计算这个 Node 要写什么（writes），读什么。在读的东西里，哪些是现成的（met_dependencies），哪些是需要其他 IR 算的（unmet_dependencies）。有了这些 SchedulerNode 后，Scheduler 会做一次拓扑排序，确保每个 Kernel 跑的时候，输入都已经算好了。这个顺序也是的 wrapper 里 kernel 运行的顺序，WrapperCodeGen 生成 xxx_kernel.run() 的代码在 /torch/_inductor/codegen/wrapper.py:L1224，每一次 xxx_kernel.run()都使用了相同的 stream。也就是说，目前 Inductor 生成的 kernel 始终是单线程/stream 运行的。就算整个函数能被拆成两个输入输出毫不相关的 kernel， Inductor 也还是会把他们放在同一个 stream 下，一个跑完了再跑另一个。

在 GraphLowering 的阶段，Inductor 会把 Pointwise 和 Pointwise，作为 Reduction 输入的 Pointwise fuse 在一起。在 Scheduler 里，Inductor 就会处理更多类型的 fusion。

Inductor Scheduler 会利用 can_fuse（/torch/_inductor/scheduler.py:L1945）先进行一些基本的 check 包括：

Extern Kernel 不能 fuse 任何 Kernel；

Template 只能 fuse 简单的 epilogues；

两个 Node 读写必须有 share 的部分；

两个 Node 必须在相同的 device 上计算。

此外还有 can_fuse_vertical()的 check，是指融合后两计算操作间的中间变量无需内存，一个位于 torch/_inductor/scheduler.py:L2030，另一个位于 torch/_inductor/codegen/triton.py:L3037，主要 check fuse 后 Inductor 是否能生成正常分块的 Triton Kernel，这里还会 check horizontal fusion，示例如下，先 load 一个[n,2,32]的 block，在最后一个 dim 做 sum，就得到了 a 对应的 tile([n,2])。再做一次 sum 就得到 b 对应的 tile([n])，a 和 b broadcast 后相加就得到了 c 对应的 tile，torch.compile 这个函数，Scheduler 会拿到 3 个 ComputedBuffer，分别对应 abc，先看 buf0 和 buf1 能不能 fuse，buf0 和 buf1 共享同样的输入，输出到不同的 tensor 里，Inductor 把这种 Fusion 称为 Horizontal Fusion。目前 Reduction-Reduction 的 fusion，要求两个 node 的 group 完全一致，所以不能自动生成最优的 Triton Kernel。只有 Reduction（buf0/1）和 Pointwise（buf2）能 fuse，这里 Inductor 有两个选择，Fusion 省下的是 IO，而 a 比 b 大，所以 fuse buf0 和 buf2 收益更大。Inductor Scheduler 的 Fusion 是贪心，get_possible_fusions（_inductor/scheduler.py:L1832）会把所有可能的 fusion 按节省的 IO 排序，并且优先一些类型的 fusion。在 fuse 了一些 pair 后，可能又会出现一些新的 fusion opportunity 。所以在 fuse_nodes 里，Inductor Scheduler 会进行最多 10 轮 fuse_nodes_once，每一轮都会重新 get_possible_fusions。

```
@torch.compile
def f(x: torch.Tensor):
    a = x.reshape([-1,2,32]).sum(dim=-1) # buf0 Reduction group=(1024, 32)
    b = x.reshape([-1,1,64]).sum(dim=-1) # buf1 Reduction group=(512,  64)
    c = a+b                              # buf2 Pointwise group=(1024, 1)
    return c
f(torch.empty(32768, device="cuda"))
```

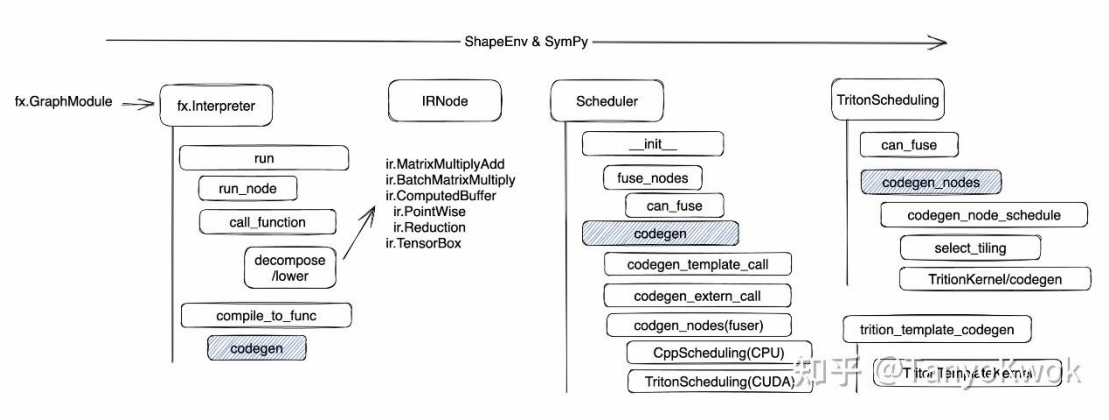
matmul 以后接一个 relu 的融合示例，matmul 既可以 lower 成 ExternKernel，也可以 lower 成 TritonTemplate。结合上面的 can_fuse 的规则，ExternKernel 什么都不能 fuse，Template Template 则可以 fuse 后面的 relu。虽然 lowering 里的选择会影响 Scheduler 里的 Fusion，Inductor 只会 benchmark 两种 matmul 性能，选择跑得快的一个。Template 的 Fusion 实现非常巧妙。matmul 的 Triton Template 里有一些{{}}包起来的函数，这些{{xxx}}最后会被 TritonTemplateKernel 里对应函数的返回值取代。其中 store_output 会用“正常”的逻辑生成最后的 tl.store。在 benchmark 只要一个 kernel 时，可能有些多此一举。但是这个要 tl.store 的值就会存在 CSE 里，对应的代码也会包在 DeferredLine 里，后面 relu 的 fusion 也是水到渠成。

```
mm_template = TritonTemplate(
    name="mm",
    grid=mm_grid,
    source=r"""
{{def_kernel("A", "B")}}
    M = {{size("A", 0)}}
    N = {{size("B", 1)}}
    K = {{size("A", 1)}}
    ...
    acc = tl.zeros((BLOCK_M, BLOCK_N), dtype=ACC_TYPE)
    for k in range(K, 0, -BLOCK_K):
        ...

    # rematerialize rm and rn to save registers
    rm = pid_m * BLOCK_M + tl.arange(0, BLOCK_M)
    rn = pid_n * BLOCK_N + tl.arange(0, BLOCK_N)
    idx_m = rm[:, None]
    idx_n = rn[None, :]
    mask = (idx_m < M) & (idx_n < N)

    # inductor generates a suffix
    {{store_output(("idx_m", "idx_n"), "acc", "mask")}}
""",
)
```

融合后的结果是 FusedSchedulerNode，其将原来的 Node 包在一块重新计算了 dependencies 和 writes，为后续的 fusion 提供信息，最后结果可以在 ir_post_fusion.txt 找到。/torch/_inductor/config.py:L93



scheduler.py

OutputNode

fuse node1: BaseSchedulerNode node2:BaseSchedulerNode

node1 和 2 有一个是 foreache，则 ForeachKernelSchedulerNode.fuse

否则，FusedSchedulerNode.fuse

BaseSchedulerNode

ExternKernelSchedulerNode

NopKernelSchedulerNode

SchedulerNode

BaseSchedulerNode.simplify_and_reorder //进行后端无关的循环变换

group

//获取内存读写依赖

```
    is_template: set_read_writes(ComputerBuffer.normalized_read_writes())
    else: dependencies.extract_read_writes()
FusedSchedulerNode
    set_read_writes()
    unmet_dependencies
    min_order
    max_order
ForeachKernelSchedulerNode
    can_fuse
        if producer.is_foreach and consumer.is_foreach:
            return len(producer.snodes)== len(consumer.snodes) and
                    all(producer.scheduler.canfuse(l,r))
        elif consumer.is_foreach:
            cosumer_subnode
            check consumer.scheduler.can_fuse(producer, cosumer_subnode)
        elif producer.is_foreach:
            producer_subnode
            check producer.scheduler.can_fuse(producer_subnode, cosumer)
```

can_fuse 用于判断是否可以融合两个 BaseSchedulerNode

如果传入的 producer 和 consumer 节点都是"foreach"节点，则会检查它们的子节点数量是否相同，相同则会继续逐个检查子节点是否可以融合。

如果 consumer 是"foreach"节点而 producer 是"reduction"节点，则无法融合，返回 False。当 consumer 是"foreach"节点而 producer 节点不是"reduction"节点时，会尝试获取 consumer 节点中与 producer 节点相关的子节点 consumer_subnode，即寻找依赖项。如果找到了 consumer_subnode，则会调用 can_fuse 方法来判断 producer 节点和 consumer_subnode 是否可以融合，没有找到则表明 producer 节点不是任何"foreach"producer 算子的依赖项，无法融合，返回 False

如果 producer 是"foreach"节点而 consumer 是"reduction"节点，则同样无法融合，返回 False。当 producer 节点是"foreach"节点而 consumer 节点不是"reduction"节点时，会尝试获取 producer 节点中与 consumer 节点相关的子节点 producer_subnode，即寻找依赖项。如果找到了 producer_subnode，则会调用 can_fuse 方法来判断 producer_subnode 和 consumer 节点是否可以融合，没有则表明 consumer 节点不是任何"foreach"producer 算子的依赖项，无法融合，返回 False。

如果以上条件都不满足，则会引发一个断言错误。

fuse 执行实际的融合操作，用于融合两个 BaseSchedulerNode，并创建一个新节点

首先通过断言确保 producer 节点和 consumer 节点中至少有一个节点是"foreach"节点。

如果 producer 和 consumer 都是"foreach"节点，则将它们的子节点逐个融合成新的节点，并将融合后的节点存储在 fused_nodes 列表中。

如果 producer 是"foreach"节点而 consumer 不是，则找到 producer 中与 consumer 相关的子节点 producer_subnode，对于 producer 的每个子节点，将找到的 producer_subnode 与 consumer 进行融合，形成新的节点，并存储在 fused_nodes 列表中。

如果 consumer 是"foreach"节点而 consumer 不是，则类似地找到 consumer 中与 producer 相关的子节点 consumer_subnode，对于 consumer 的每个子节点，将 producer 与找到 consumer_subnode 进行融合，形成新的节点，并存储在 fused_nodes 列表中。

最后，根据融合操作的情况，返回一个新的 ForeachKernelSchedulerNode 实例，其中包含了原始调度器 producer.shceduler、融合后的节点列表 fused_nodes 以及相关的 prev_node_1 和 prev_node_2 信息。

fuse_nodes_once 用于将节点融合为 FusedSchedulerNodes

首先通过 get_possible_fusions()方法获取可能的节点融合组合并逐一进行判断，对每组节点 node1 和 node2 通过 can_fuse 和 will_fusion_create_cycle 判断是否能融合以及融合后两节点间是否不会产生环，如果是则通过 speedup_by_fusion 判断融合后是否能够提升性能，如果是则执行融合操作，在融合过程中，调用 fuse()生成一个新的融合节点 node3，将 node1 和 node2 从融合节点集合中移除，将 node3 添加到集合中，并更新节点名称映射关系。

最后，将融合后的节点集合 fused_nodes 按照 min_order 进行排序，调用 topological_sort_schedule 进行拓扑排序，并调用 prune_redundant_deps 修剪冗余依赖关系。通过这些步骤，实现了一次节点融合的操作。

get_possible_fusions 用于查找所有可能的合法融合机会，并按照 score_fusion 进行评估融合方案进行排序

首先初始化一个空的 possible_fusions 列表和一个空的 seen 集合。

然后根据节点使用的缓冲区名称，将节点分成不同的组，并在每个组上调用 check_all_pairs 方法，以找到可能的融合机会。

check_all_pair 方法中将输入的节点列表进行两两组合（seen 用于判断是否枚举完成所有组合），并调用 can_fuse 检查它们是否可以进行融合。如果两个节点可以融合，则将它们组成一个元组，添加到 possible_fusions 列表中；如果 node2 是 template 节点或 foreach 节点，并且可以与 node 1 融合，则也将其组成一个元组添加到 possible_fusions 列表中。

如果启用了 config.aggressive_fusion，则将节点按照 group 属性分组，并对每个组调用 check_all_pairs 方法。

最后，对得到的所有可能融合机会 posible_fusions 调用 get_possible_fusions_with_highest_priority 根据来自后端的优先级对可能的融合进行分组，筛选出具有最高优先级的一组，并按照 score_fusion_key 优先级排序。

score_fusion_key 调用 score_fusion，score_fusion 返回一个四元组：

第一个布尔值表示 node1 是否是一个 template 节点且配置中指定 config.epilogue_fusion_first，且 memory_score 大于 0

第二个布尔值表示 node1 和 node2 是否都是 reduction 节点，并且 memory_score 大于 0

第三个 memory_score 调用 score_fusion_memory 估计 node1 和 node2 融合后能节省的内存操作量

第四个 proximity_score 评估两个节点在原始序列中接近程度

will_fusion_create_cycle 用于判断是否存在由其他节点融合间接导致的从 node1 到 node2(或者反之)的路径

首先初始化一个空的 visited 集合，之后计算两节点祖先节点去除本身的并集 combined_ancestors，之后调用 found_path 判断是否融合会引入环，具体地，检查融合节点是否是 node1 和 node2 祖先节点 combined_ancestors 的子集，如果是则返回 False，否则

继续对融合节点引入的新祖先节点进行深度优先搜索，查找是否会引入环。

speedup_by_fusion 用于判断 node1 和 node2 节点融合是否会提升性能

如果配置 config.benchmark_fusion 为 False，且 node1 不是 template 或 MultiTemplateBuffer 的实例，则返回 True

如果 node1 是 template 但不是 ir.TritonTemplateBuffer 类型，或者 node1 是 foreach 节点，或者 node2 是 foreach 节点，则返回 True

如果 node1 的设备类型为 CPU 则返回 True

如果融合节点中包含有通过 atomic_add 执行的 kernel，则返回 True

如果 node1 是否为 SchedulerNode 类型且 node1.node 是否为 ir.MultiTemplateBuffer 类型。

如果满足条件，获取 multi_node 中的 choice_timings，对 node_list_2 中的节点进行性能基准测试，遍历 choice_timings 中的选择，如果选择满足条件，则执行节点融合操作，并记录性能，根据性能结果选择是否保留融合后的节点作为新的模板。

如果不满足条件则对 node_list_1、node_list_2 和融合后的节点进行性能基准测试，并处理可能的异常情况。如果出现编译错误，根据错误信息判断是否允许融合操作。记录融合的性能情况，并根据一定条件记录慢融合情况。

class Scheduler

can_fuse 用于判断是否可以将 node1 和 node2 合并成一个融合节点

首先判断 node1 和 node2 是否相同，相同则无法融合，返回 False

如果 node1 是 ExternKernelSchedulerNode 或 NopKernelSchedulerNode 节点，而不是 template 则无法融合，返回 False

如果 node2 是 ExternKernelSchedulerNode 或 NopKernelSchedulerNode 节点，而不是 template 则无法融合，返回 False

如果 node2 在 node1 之前执行则无法融合，返回 False

如果 node2 是 template，由于 template 仅能融合 epilogues，故无法融合，返回 False

如果 node1 是 template，node2 是 has_aliasing_or_mutation、reduction、config.epoligue_fusion 设置为 False 这三个条件之一，则不满足 template epilogue 条件，无法融合

如果 node1 和 node2 的设备不匹配，则无法融合，返回 False。

如果没有共享数据（score_fusion_memory(node1,node2)==0）且 config.aggressive_fusion 为 False 或 node1 是 reduction 或 node2 是 reduction，则无法融合返回 False

如果 node1 和 node2 节点数量之和大于 config.max_fusion_size 且 node1 和 node2 不是 foreach 节点，则超过最大融合限度而无法融合，返回 False

如果 node1 与 node2 的祖先相交，则调用 can_fuse_vertical 进行垂直融合判断，否则调用位于 TritonScheduling 中的 can_fuse_vertical；否则调用 can_fusion_increase_peak_memory 判断融合会增加峰值内存（由于原本在原始顺序中相隔较远的节点被融合后，会延长张量的生命周期，从而增加内存占用，这里设置经验阈值 64），如果增加则返回 False，否则调用位于 TritonScheduling 中的 can_fuse_horizontal 进行水平融合判断。

score_fusion_memory 用于计算两节点融合可以节省的内存操作量

首先通过获取两节点读写内存地址集合的并集后，求两个并集的交集来确定共同的内存依赖，即两节点都访问的内存地址 common_memory_deps，之后遍历 common_memory_deps 调用 dep_size_hint 获取内存操作资源量并进行求和后返回。

can_fuse_vertical 用于检查是否能够将 consumer 节点 node2 融合至 producer 节点 node1 中。

首先使用集合 computed_deps 来记录 node2 中未满足的依赖 unmet_dependencies 可以从 node1 的写入操作中得到满足依赖项，具体地，通过遍历 node1 的写入操作和 node2 的未满足依赖 unmet_dependencies, 并调用 fusable_read_and_write 方法来判断的。

然后检查 node2 的剩余依赖，即从 node2 的未满足依赖 unmet_dependencies 中去除已经在 computed_deps 中的依赖，得到剩余的依赖 remaining_deps。

如果 remaining_deps 中包含 node1 的缓冲区名称，这意味着 node2 试图读取 node1 中未写入的缓冲区位置，或者读取了不同的缓冲区（如不同的索引或使用了不同的依赖类型），这是不允许的，无法融合，返回 False。

之后检查中间节点，对于 remaining_deps 中的每个依赖，检查其定义操作 defining_op 的祖先节点是否包含 node1 的操作。如果包含，则意味着在 node1 和 node2 之间存在中间节点，这可能会破坏融合的合法性，无法融合，返回 False。

最后检查写入和读取的对齐，遍历 node2 的写入操作和 node1 的读取操作，检查写入操作的名称是否与读取操作的名称相匹配，调用 fusable_read_and_write 以检查两种类型 StarDep 和 MemoryDep 读写操作的索引和大小是否相同，是否能够融合。

TritonScheduling 中的 can_fuse_horizontal 和 can_fuse_vertical，位于 torch/_inductor/codegen/triton.py，均在最后调用该类下的 can_fuse 实现

can_fuse 用于判断两个节点是否可以被 Triton 后端融合

首先检查节点类型，如果两个节点都是 ForeachKernelSchedulerNode 类型的节点，则调用该类型的 can_fuse 方法来判断是否可以融合。

检查是否是 reduction 节点，如果两个节点都是 reduction 节点，则检查它们的元素数量 numel 和 reduction 元素数量 rnumel 是否相同。

如果两个节点都不是 reduction 节点，则同样检查元素数量 numel 和 reduction 元素数量 rnumel 是否都相同，如果均不相同则返回 False，并且如果 node1 是模板节点，则跳过后续检查，返回 True。

然后检查融合时的 Tiling 来判断是否可以进行融合，如果配置中指定了 tiling_prevents_pointwise_fusion，则需要进一步比较 tiling1（node1）、tiling2（node2） 和 tiling3（node1+node2） 中 tiling 长度大于 2 的 tiling 是否相等。如果没有启用 tiling_prevents_pointwise_fusion，则默认可以融合。

接下来处理非 reduction 和 reduction 节点的融合，如果 node1 是 reduction 节点而 node2 不是，rnumel1==1，rnumel2!=1，且 numel1==numel2 * rnumel2 相等。同时，检查非归约节点 node1 下的每个子节点的范围是否与归约节点 node2 的输出范围兼容。如果启用了 tiling_prevents_reduction_fusion 并且非归约节点 node1 不是 template 节点，则需要检查非归约节点的 tiling 是否满足特定条件(numel1, 1)或者(numel2, rnumel2, 1)。

如果 node1 是 reduction 节点而 node2 不是，则交换这两个节点的参数，调用 can_fuse_horizontal 方法重新进行判断。can_fuse_vertical 和 can_fuse_horizontal 被设置为 can_fuse 的别名。